

WORK UPDATE

Name: D Sri Ramya

Date:25-2-2026

Project Overview & System Architecture

1. Introduction

This project is a scalable food delivery platform inspired by Zomato. The system consists of four separate applications:

- Customer App
- Restaurant App
- Driver App
- Admin Panel

The system enables customers to browse restaurants, place orders, make payments, and track deliveries in real-time. Restaurants manage orders and menus. Drivers handle pickup and delivery. Admin monitors and manages the entire platform.

2. Objectives

- Provide seamless food ordering experience
 - Enable real-time order tracking
 - Support secure online payments
 - Maintain scalable microservices architecture
 - Ensure role-based access control
-

3. High-Level Architecture

The system follows a **Microservices Architecture** pattern.

Architecture Layers:

Client Layer

- Customer Mobile App
- Restaurant App
- Driver App

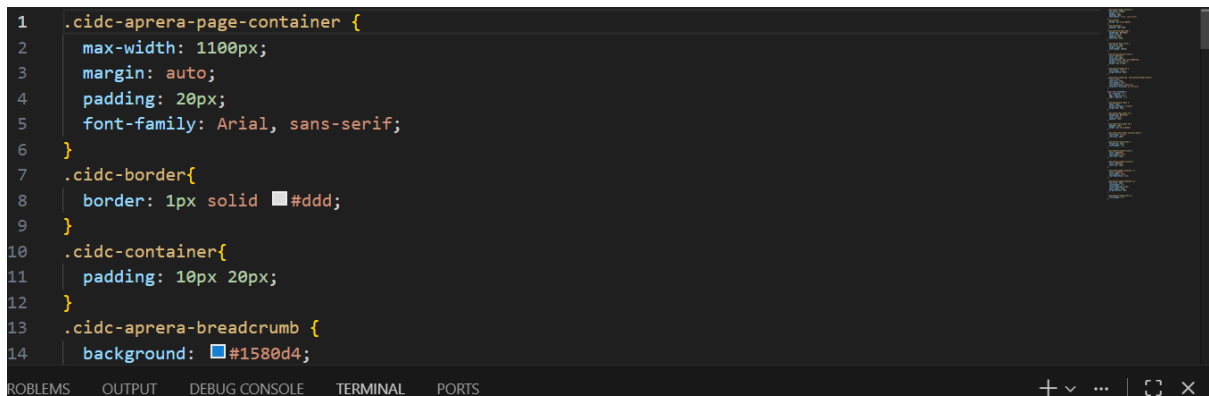
WORK UPDATE

- Admin Web Dashboard

API Gateway

- Central entry point
- Authentication validation
- Request routing

```
1  .cidc-aprera-page-container {
2    max-width: 1100px;
3    margin: auto;
4    padding: 20px;
5    font-family: Arial, sans-serif;
6  }
7  .cidc-border{
8    border: 1px solid #ddd;
9  }
10 .cidc-container{
11   padding: 10px 20px;
12 }
13 .cidc-aprera-breadcrumb {
14   background: #1580d4;
```



Microservices Layer

- Auth Service
- User Service
- Restaurant Service
- Order Service
- Payment Service
- Delivery Service
- Notification Service

Database Layer

- MongoDB / PostgreSQL (Main Data)
- Redis (Caching & Real-time data)
- Firebase (Push Notifications)

4. Technology Stack

Frontend:

- React (Admin Panel)

WORK UPDATE

- React Native (Mobile Apps)

Backend:

- Node.js
- Express.js
- JWT Authentication
- Socket.io (Real-time tracking)

Database:

- MongoDB
 - Redis
-

System Flow & Functional Modules

1. Customer Flow

1. User Registration / Login
 2. Location detection
 3. Browse nearby restaurants
 4. View menu
 5. Add items to cart
 6. Place order
 7. Online payment
 8. Track order live
 9. Receive delivery
-

2. Restaurant Flow

1. Login
2. Receive new order notification
3. Accept / Reject order
4. Prepare food

WORK UPDATE

5. Mark order as ready
6. Handover to driver

3. Driver Flow

1. Login
2. View assigned orders
3. Accept delivery request
4. Pickup from restaurant
5. Deliver to customer
6. Mark as delivered

I worked on designing the overall architecture and low-level design for our Zomato-like food delivery system, including the four modules: Admin, Restaurant, Customer, and Driver. I finalized the system flow, order lifecycle, database schema, and core class structure. I also defined the microservices architecture and real-time tracking approach. Today, I will start implementing the backend structure and API design for the order and authentication modules. Currently, there are no blockers from my side